

Can Interpretability-Guided Training Reduce JetClass-to-AspenOpenJets Domain Shift?

Ryan Yang
Brown University

April 30, 2026

Abstract

Transformer-based jet taggers perform very strongly on simulated datasets, but their behavior under the simulation-to-real-data shift is less certain. This project studies whether interpretability-guided training is able to reduce the domain shift between a JetClass-trained tagger and the unlabeled AspenOpenJets dataset while still preserving most of the JetClass performance. I trained a 10-class transformer jet classifier on JetClass and calibrated the unlabeled shift metrics on controlled corruptions of the labeled JetClass test set, evaluated the trained models on AspenOpenJets, and then tested a right-for-the-right-reasons style of intervention that penalizes in-training input gradients on three interpretability selected features. Across five baseline model seeds, the corrected JetClass mode got 0.7459 ± 0.0073 accuracy and 0.9651 ± 0.0019 macro AUC. AspenOpenJets evaluation showed a big shift in the model’s prediction distribution. In a focused 24 configuration interpretability-guided training sweep, all of the configurations improved the main Aspen shift summary relative to the baseline model. The best configuration reduced the strong shift summary metric from 0.6199 to 0.4701, at a bit of a cost of reducing JetClass accuracy from 0.7425 to 0.6937. The result supports the main hypothesis qualitatively: attribution-guided training can change a model’s reliance in a way that reduces the unlabeled target-domain shift, but it does introduce a non-trivial accuracy-robustness tradeoff.

Codebase

The project code is available at https://github.com/ryanlyang/CompPhys_Final. The main final-project implementation (training/evaluation scripts, AspenOpenJets utilities, RRR find-another runs, aggregations, and plotting) is in `project_pipeline/`. Supporting baseline transformer code is in `jetclass_transformer/`, with most of that folder inherited from the public Particle Transformer implementation [1].

All of the code I wrote for this final project is in `project_pipeline/`. Some files in that folder include a mix of new code and inherited/adapted components from earlier research-group work, especially `project_pipeline/train_eval_jetclass_canonical_aspen.py` and `project_pipeline/reimplement_preliminary_studies.py`. For more info see the repository `README.md`.

1 Introduction

My precise computational physics question for this project is:

If a transformer jet tagger is trained on labeled simulated JetClass jets, can right-for-the-right-reasons gradient regularization reduce its prediction-distribution shift on unlabeled real CMS-derived AspenOpenJets jets, as measured by calibrated output-space shift diagnostics, without destroying its held-out JetClass accuracy?

To answer this question I trained JetClass classifiers, stress-tested them with controlled corruptions compared to known accuracy metrics, and used those comparisons to rank unlabeled shift metrics, and then evaluated the same models on AspenOpenJets and compared the baseline models to the attribution-guided retrained models on the joint tradeoff between JetClass test accuracy and AspenOpenJEs shift.

Jet tagging is the task of identifying the particle or the decay process that initiated a reconstructed jet. It is very central to LHC analyses because many signatures of Higgs bosons, electroweak bosons, top quarks, and new particles appear as jets with different internal makeups [2, 3]. Modern deep learning models, especially the particle-cloud and transformer architectures perform incredibly well on simulated jet benchmarks [1]. However, a strong model on simulated data can learn features that are specific to the simulation, the reconstruction in the simulation, or the labeling conventions, rather than the features that will actually transfer to real data.

This project looks into that issue computationally. JetClass gives us a large labeled simulated benchmark for 10-class jet tagging training [1]. AspenOpenJets gives us a large machine learning ready dataset that is derived from the CMS 2016 Open Data [4]. AspenOpenJets is not labeled in a way that gives direct supervised accuracy, so the project cannot simply train on one dataset and report accuracy on the other. Instead, I first calibrated unlabeled shift metrics on the controlled JetClass corruptions, where the labels are known, and then I used those metrics on AspenOpenJets as the target-domain diagnostics.

I discovered a major issue between the preliminary results and this final draft. The preliminary results were being affected by a labeling and evaluation mismatch in the JetClass dataset that made metrics and results unreliable. The final results in this paper come from a corrected reimplementation with explicit label-mapping checks, replay tests, masking parity checks, and five-seed averaging (for the baseline). This is why some of the results here differ from the preliminary results.

2 Background

2.1 Jet Tagging and Modern Jet Representations

Jet Tagging is quite difficult because the objects being classified are not the direct detector hits but rather clustered sprays of final-state particles that are produced by parton showering and hadronization. In boosted regimes the decay products of heavy objects such as W , Z , Higgs, or top particles can end up merging into one jet, so the relevant information is encoded in the jet’s internal substructure. Reviews of the jet substructure emphasize that the useful taggers have to be able to separate the physically meaningful radiation patterns from ordinary QCD jets while they remain robust to detector effects, pileup, and simulation mismodeling [2, 3].

Before deep learning became quite dominant, jet substructure taggers often used physics-motivated observables. N -subjettiness measures how consistent a jet is with a chosen number of hard subjects, making ratios like τ_{21} and τ_{32} useful for boosted boson and top tagging [5, 6]. Soft Drop grooming removes the soft wide-angle radiation and helps stabilize the jet mass and the related observables against contamination from the underlying even and pileup [7]. These ideas matter here because they help define what a transferable jet feature should look like: that robust substructure

information should be more reliable under the simulation-to-real-data shift than the detector or generator specific artifacts.

Machine learning jet taggers have progressed from hand-designed observables and jet images towards constituent-level architectures. Jet-image studies show that computer vision methods could learn discriminants from calorimeter-like representations [8, 9]. Later models used particle constituents more directly. Set-based architectures such as Energy Flow Networks and Particle FLOW Networks helped make permutation-invariant particle representations central within the field [10]. ParticleNet then treated jets like they were particle clouds and used dynamic graph convolutions to achieve strong performance on benchmark tagging tasks [11]. The Particle Transformer further improved the constituent-level tagging by combining attention with pairwise particle-interaction information [1]. All of these developments motivate the model choice in this project, a transformer-style of classifier can learn all the strong in-domain JetClass discrimination but its flexibility also makes it able to latch on to the simulation specific shortcuts.

2.2 JetClass and AspenOpenJets as Source and Target Domains

JetClass is a huge simulated benchmark with 10 jet categories, including the Higgs, vector boson, top, and quark/gluon jets [1]. The full public benchmark is much larger than the subset I used for this project, it has 100 million training jets, 5 million validation jets, and 20 million test jets. The original Particle transformer study reported very strong results on this benchmark, showing that the constituent-level attention models are able to be incredibly effective when trained and tested within the same simulated domain.

AspenOpenJets plays a different role here. It contains 178 million high- p_T AK8 jets derived from CMS 2016 Open Data and it is geared towards machine-learning use [4]. It includes constituent-level information and higher-level jet substructure quantities, but it is not a supervised direct accuracy benchmark. It is useful here because it gives a real-data target domain rather than just another simulated benchmark. It is also intrinsically harder to quantify performance on: the jets are not labeled and the detector/reconstruction chain differs from the fast-simulation setting that is used by JetClass.

The pairing of these two datasets gives us a strong source-target setup rather than a standard supervised benchmark. JetClass gives us labels and controlled training space, while AspenOpenJets gives us realistic but unlabeled target data. The goal is therefore not to just maximize JetClass accuracy, but to ask whether the trained model’s behavior remains stable and strong when applied to the real CMS-derived jets. This is what motivates using the unlabeled prediction-space diagnostics and treating any AspenOpenJets result as a shift measurement rather than trying to measure direct accuracy.

2.3 Interpretability Methods

I used three attribution/interpretability methods here. Input gradients measure the derivative of a chosen model score with respect to its input features, which follows the general saliency-map idea that local sensitivity can be interpreted as a feature-importance signal [12]. Integrated Gradients average input gradients along a path from a baseline input to the real input [13]. SmoothGrad averages the noisy input-gradient explanations to reduce high-frequency attribution noise [14]. In a constituent-level jet model these methods will ask related but distinct questions, specifically which particle features locally affect the score the most, which features matter relative to a baseline, and which sensitivities stay stable under local noise.

The training intervention is based on the right-for-the-right-reasons (RRR) idea of penalizing

gradients in regions where the model should not rely strongly [15]. Follow up work on explanation penalties and attribution priors have shown that attributions can be used not only to audit models, but also to impose some prior knowledge during training [16, 17]. Here, instead of direct expert annotations, the masks are being constructed from domain-prior features and then expanded and increased using model attributions from previous training iterations. This turns interpretability into an actual training signal, the model is encouraged to find alternative explanations that rely less on the previously identified features.

Previous research in other fields are not automatic evidence that every attribution method will be faithful. Raw gradients are often noisy or saturated, Integrated Gradients tend to depend on a baseline choice, and SmoothGrad improves local stability but doesn’t fully guarantee casual importance. For that reason I treated attribution quality as something to test rather than assume it is faithful. The targeted versus random masking study in the results section checks whether these attribution-selected features affect the model more than randomly selected features at the same masking budgets.

2.4 Unlabeled Domain Shift

The theoretical domain-adaption setup explains why AspenOpenJets is hard to evaluate. Target error depends on source-domain performance and also on the discrepancy between source and target distributions and on whether there exists some model that performs well on both domains [18]. Domain-adversarial training is a common response to this problem, it tries to learn representations that are predictive for the source task while being less informative about which domain an example actually came from [19]. This project takes a different route. Rather than training directly on AspenOpenJets, I am using source-side corruptions and explanation-guided penalties to test out whether the model can be made less sensitive to source-target mismatch without ever training on the target.

In this project, the target dataset is unlabeled, so I can’t directly estimate target error. Instead I can measure how much the trained model’s prediction distribution changes between clean JetClass and AspenOpenJets. The broader dataset-shift literature cautions that detecting a shift and predicting its performance impact are very different tasks. Shift detection studies show that black-box prediction summaries can reveal distribution changes, but they do not always map reliably to accuracy loss [20, 21]. Uncertainty and confidence metrics are also known for behaving differently under dataset shift than under in-domain testing [22]. This is why the final analysis reports unlabeled shift metrics. I tested pseudo-accuracy conversion but eventually rejected it as it was too extrapolation-sensitive.

3 Methods

3.1 Baseline Classifier

The baseline is a particle transformer classifier that is trained on JetClass constituent-level-features. Each run used 150,000 training jets, 50,000 validation jets, and 150,000 test jets, with up to 128 constituents per jet. All JetClass training, RRR training, and AspenOpenJets inference jobs were run on RIT Research Computing Services A100 GPU resources [23]. The model was trained with 10-class cross entropy,

$$L_{\text{CE}} = -\frac{1}{N} \sum_{i=1}^N \sum_{c=1}^C y_{ic} \log \hat{p}_{ic}, \quad (1)$$

where y_{ic} is the one-hot class label and $\hat{p}_{ic}$ is the predicted probability for class c .

The clean baseline was run for five random seeds. For the final analysis, trainer and post-training inference metrics were checked for exact consistency after checkpoint reload.

3.2 Unlabeled Shift Metrics

For a dataset A , I define the mean predicted class-probability vector as

$$\bar{p}_A = \frac{1}{N_A} \sum_{i \in A} \hat{p}(x_i). \quad (2)$$

For Clean JetClass predictions and shifted predictions I measured five unlabeled diagnostic: probability L^1 drift, top-1 prediction histogram drift, Jensen-Shannon divergence of the mean class distributions [24], confidence drop, and entropy shift. Predictive entropy is

$$H(x) = - \sum_{c=1}^C \hat{p}_c(x) \log \hat{p}_c(x). \quad (3)$$

The strongest three metrics in the final analysis were probability L^1 drift, top-1 drift, and class-distribution Jensen-Shannon divergence. I report their average as the ‘strong3’ shift summary in the intervention study.

3.3 Controlled Corruptions

To validate the unlabeled shift metrics for AspenOpenJet use, I applied controlled corruptions to the JetClass test inputs and compared each unlabeled metric to the true supervised degradation. The corruptions included transverse-momentum noise, angular jitter, constituent dropout, merging-like effects, and global scale changes. Because JetClass labels are actually known, each corruption gives both an unlabeled shift value and a true ΔAUC or $\Delta\text{accuracy}$. This allows the shift metrics to be ranked based on correlation with AUC or accuracy before they are used on unlabeled AspenOpenJets.

3.4 Interpretability-Guided Training

For the RRR-style training [15], I used

$$L = L_{\text{CE}} + \lambda_{\text{RRR}} \|A \odot \nabla_x s_\theta(x)\|_2^2, \quad (4)$$

where $s_\theta(x)$ is the model score that was used for the gradient penalty, A is binary mask that selects the features to discourage reliance on, and λ_{RRR} controls the strength of the penalty during training.

The intervention was in a ‘find another explanation’ style of training. The goal is to repeatedly penalize features that the model previously relied on, which forces the next model to solve the task using different evidence. For each hyperparameter configuration, I ran the following procedure for five iterations:

1. Build an initial token mask by selecting the top `mask_frac` constituents per jet ranked by constituent p_T . Convert this to a feature mask only on domain-prior channels that we expect to be less stable on AspenOpenJets. Channels in this category include constituent charge, particle-type indicator channels (`is_ch`, `is_nh`, `is_pho`, `is_ele`, `is_mu`), and track-displacement features such as `d0`, `d0err`, `dz`, and `dzerr`.

2. Train iteration-1 model from scratch with the RRR loss above using that mask.
3. For iteration $k \geq 2$, compute attributions for every training jet using the previous model (source = input gradients, Integrated Gradients [13], or SmoothGrad [14]).
4. For each training jet, select the top `mask_frac` constituents by attribution magnitude and convert them to a full-feature token mask. Update the cumulative mask by logical union with all previously selected masks.
5. Train iteration- k model from scratch using the updated cumulative mask.
6. After each iteration, evaluate JetClass test metrics and run AspenOpenJets inference, then continue to the next iteration.

This means that a single hyperparameter point is not one training run, it is five full retrainings, four full-training-set attribution passes (iterations 2-5) and five full evaluation passes. I initially intended to run a broader sweep including more λ_{RRR} values but this nested computation cost made the full grid expensive to run in a reasonable amount of time. So the final reported sweep covers $\lambda_{\text{RRR}} \in \{1, 10\}$ and mask fractions $\{0.05, 0.10, 0.20, 0.30\}$ for all three of the attribution sources, giving us 24 focused hyperparameter configurations.

4 Reliability Checks

Several sanity checks were built into the final pipeline.

First, all clean JetClass results were rerun over five seeds (fixed from preliminary results). Second, the checkpoint replay was tested by reloading the best checkpoint and reproducing validation/test behavior. Third, the class balance, label ordering, and split overlap were checked to be consistent. Fourth, the interpretability masking was tested against random masking at exactly matched masking budgets. Fifth, AspenOpenJets inference was sanity-checked for normalized and finite probability outputs, expected sample size, and consistently with the stored clean/corruption references.

The most important limitation is that AspenOpenJets doesn't give labels. Therefore, no claim in this paper should be read as a measured AspenOpenJets accuracy. I tried to calibrate the unlabeled shift metrics into a pseudo-accuracy domain drop using the controlled JetClass corruptions, but the conversion was clearly flawed, made too many assumptions, and was not used as a reported result. The Aspen results should instead be read as risk proxy diagnostics. The sanity checks support that the pipeline is functioning correctly but they don't remove the fundamental uncertainty of evaluating real CMS derived data without labels.

5 Results

5.1 Clean JetClass Baseline

The corrected baseline runs completed for all five seeds. Table 1 shows the aggregate performance. The macro one-vs-rest AUC is stable across seeds, and the checkpoint replay difference is exactly zero in the stored diagnostics, indicating that the training-time and reloaded-evaluation paths agree.

Table 1: Clean JetClass performance aggregated over five seeds.

Metric	Mean $\pm$ Std	Min	Max
Accuracy	0.7459 ± 0.0073	0.7369	0.7551
Macro AUC (OvR)	0.9651 ± 0.0019	0.9628	0.9674
Signal-vs-background AUC	0.9573 ± 0.0015	0.9552	0.9587
Signal-vs-background FPR@50% TPR	0.00785 ± 0.00050	0.00727	0.00840
Target-vs-background AUC	0.9910 ± 0.0023	0.9877	0.9934
Target-vs-background FPR@50% TPR	0.000747 ± 0.000328	0.000333	0.001067

Table 2: Unlabeled shift-metric calibration against corruption-induced Δ AUC.

Metric	Spearman(Δ AUC)	Pearson(Δ AUC)
prob_l1_drift	0.9567 ± 0.0149	0.9851 ± 0.0009
top1_flip_rate	0.9567 ± 0.0149	0.9849 ± 0.0010
class_js_divergence	0.9567 ± 0.0149	0.9296 ± 0.0072
entropy_shift	0.8467 ± 0.0701	0.6443 ± 0.1767
confidence_drop	0.7600 ± 0.1512	0.4313 ± 0.2593

5.2 Corruption Calibration of Unlabeled Shift Metrics

The controlled corruption study tested whether the unlabeled diagnostics track the true supervised degradation well. Table 2 shows that probability L^1 drift, top-1 drift, and Jensen-Shannon class-distribution shift are the strongest metrics. Their spearman correlations with Δ AUC are nearly identical because the corruption severities are ordered similarly by all three metrics. Probability L^1 drift ranked first in all five seeds.

5.3 Interpretability Sanity Check

The attribution methods were tested by masking the highest-attribution constituents and comparing them against random masking using the same average number of removed constituents in both. Table 3 shows that all three of the attribution methods produce a clear targeted-versus-random gap. Integrated gradients has the largest probability-drop gap, while Smoothgrad and input gradients also identify the most relevant constituents effectively.

5.4 AspenOpenJets Shift

Each of the five baseline seed models were evaluated on 10^6 AspenOpenJets jets. Table 4 shows the shifts across all seeds. The reported Aspen result is the direct unlabeled shift numbers themselves, not a pseudo-accuracy estimate.

This is an important distinction. The shift metrics remain useful though because they quantify how much the model’s prediction distribution changes on the real CMS-derived jets, and the strongest metrics were validated against labeled JetClass corruptions. However, the separate ‘unlabeled shift metric $\rightarrow$ accuracy shift’ mapping wasn’t stable enough to include quantitatively.

Table 3: Interpretability effectiveness under matched targeted and random masking. The drop is the decrease in true-class probability averaged over mask fractions.

Method	Targeted drop	Random drop	Gap
Input gradients	0.1951 ± 0.0182	0.0305 ± 0.0011	0.1645 ± 0.0178
Integrated Gradients	0.2591 ± 0.0130	0.0305 ± 0.0011	0.2286 ± 0.0125
SmoothGrad	0.2021 ± 0.0117	0.0305 ± 0.0011	0.1716 ± 0.0118

Table 4: AspenOpenJets unlabeled shift diagnostics over five seeds.

Metric	Mean $\pm$ Std	Min	Max
<code>prob_l1_drift</code>	0.9839 ± 0.1026	0.8878	1.1494
<code>top1_flip_rate</code>	0.7123 ± 0.0298	0.6656	0.7374
<code>class_js_divergence</code>	0.1871 ± 0.0322	0.1404	0.2222
<code>confidence_drop</code>	0.2351 ± 0.0514	0.1481	0.2803
<code>entropy_shift</code>	0.6920 ± 0.1470	0.4870	0.8421

5.5 Aspen Pipeline Sanity Checks

Table 5 summarizes the checks that were used to rule out simple implementation failures. Clean metric replay, corruption replay, and Aspen Output integrity all passed for all five of the seeds. These checks made it unlikely that the large Aspen shift is only a checkpoint-loading, label ordering, probability normalization, or data format artifact. They show that the pipeline is internally consistent and that the remaining uncertainty comes mainly from the lack of supervised AspenOpenJets labels or an equally strong target-domain validation proxy.

5.6 Interpretability-Guided Training

The RRR style of training study used a focused 24 configuration sweep, three attribution-mask sources, two regularization strengths $\lambda_{\text{RRR}} \in \{1, 10\}$, and four mask fractions $\{0.05, 0.10, 0.20, 0.30\}$. I initially intended to run a wider hyperparameter sweep, but the larger grid did not prove doable on the project timescale and resources. I also did not rerun the setups for more than one seed as I did for the baseline, as that was also a very expensive cost. I also did not use Integrated Gradients or SmoothGrad directly as the differentiable in-training regularizer. Doing so would require many more gradient evaluations inside each training step, which was just too expensive for my compute resources. Instead, the actual RRR penalty used input gradients, while Integrated Gradients and Smooth Grad were still implemented as attribution methods for constructing the masks used between find-another iterations.

For comparison, the 5-seed baseline has JetClass accuracy as 0.7459, macro AUC 0.9638, and strong3 Aspen shift 0.6199. All 24 configurations in the focused sweep reduced strong3 relative to this baseline. Figure 1 shows the accuracy-shift tradeoff: the RRR runs move downward in Aspen strong3, but also move leftward in JetClass accuracy. Table 6 reports the six focused-sweep configurations that have the lowest AspenOpenJets strong3 values, while Table 7 reports the best configuration for each of the attribution-mask source methods. Within each of the runs, the reported interaction is selected by minimum AspenOpenJets strong3 shift, not by JetClass accuracy. The tables then report the corresponding JetClass test accuracy for that selected iteration.

The best focused-sweep configuration reduced the strong3 metric by about 24% relative to the

Table 5: Core AspenOpenJets sanity-check outcomes across five seeds.

Check	Pass count
Clean replay accuracy consistency	5/5
Clean replay AUC consistency	5/5
Corruption replay consistency	5/5
Aspen finite/normalized/size checks	5/5
Strong-metric mapping R^2 threshold	5/5
Strong-metric mapping intercept sanity	5/5

Table 6: Top six focused-sweep RRR configurations ranked by AspenOpenJets strong3 shift, with the baseline included for reference. The listed iteration is selected by minimum AspenOpenJets strong3 shift within that run. JetClass test accuracy is measured on the held-out JetClass test set; AspenOpenJets remains unlabeled. Strong3 is the average of `prob_l1_drift`, `top1_flip_rate`, and `class_js_divergence`; lower is better.

Mask source	λ	Mask frac.	Best iter.	JetClass test acc.	Aspen strong3
Baseline	—	—	—	0.7459	0.6199
SmoothGrad masks	10	0.05	5	0.6937	0.4701
Input-gradient masks	10	0.05	1	0.7029	0.4918
Integrated-gradient masks	10	0.05	1	0.7011	0.4998
SmoothGrad masks	10	0.10	3	0.6922	0.5004
Input-gradient masks	1	0.30	3	0.7118	0.5113
Input-gradient masks	10	0.10	1	0.7036	0.5174

baseline, but it reduced JetClass accuracy by about 4.9 percentage points. A more balanced input gradient configuration such as $\lambda = 1$ and mask fraction 0.30, reduced strong3 by about 17.5% while reducing JetClass accuracy by about 3.1 percentage points. These results show that the intervention is not just random regularization, it consistently changes the model in the direction of lower AspenOpenJets shift, while showing a non-negligible performance trade.

This improvement should be interpreted as a reduction in the unlabeled prediction-distribution shift, not as a direct measurement of AspenOpenJets accuracy. The metric is useful because it was calibrated against controlled JetClass corruptions and passed the pipeline sanity checks, but it is still an imperfect proxy for real-data performance. Confirming that the same intervention significantly improves the real collision-data classification would require a larger study with different kinds of data, stronger target-domain metrics, and more research compute than was available for the scope of this project.

Figure 2 shows the same focused sweep organized by hyperparameter cell. The best cell uses SmoothGrad masks with $\lambda_{\text{RRR}} = 10$ and mask fraction 0.05.. The pattern should not be interpreted as a global best, since larger and more diverse λ_{RRR} values were left for future work, but it does show that the improvement is not isolated to a single random run.

Figure 3 show the iteration by iteration behavior for the representative runs. The trajectories aren’t monotonic, adding more attribution derived mask coverage can improve Aspen strong3 in one iteration and worsen it in another, this supports selecting the iteration by a strong metric, while also showing why the method should be treated as a tradeoff rather than a guaranteed robustness and improvement procedure.

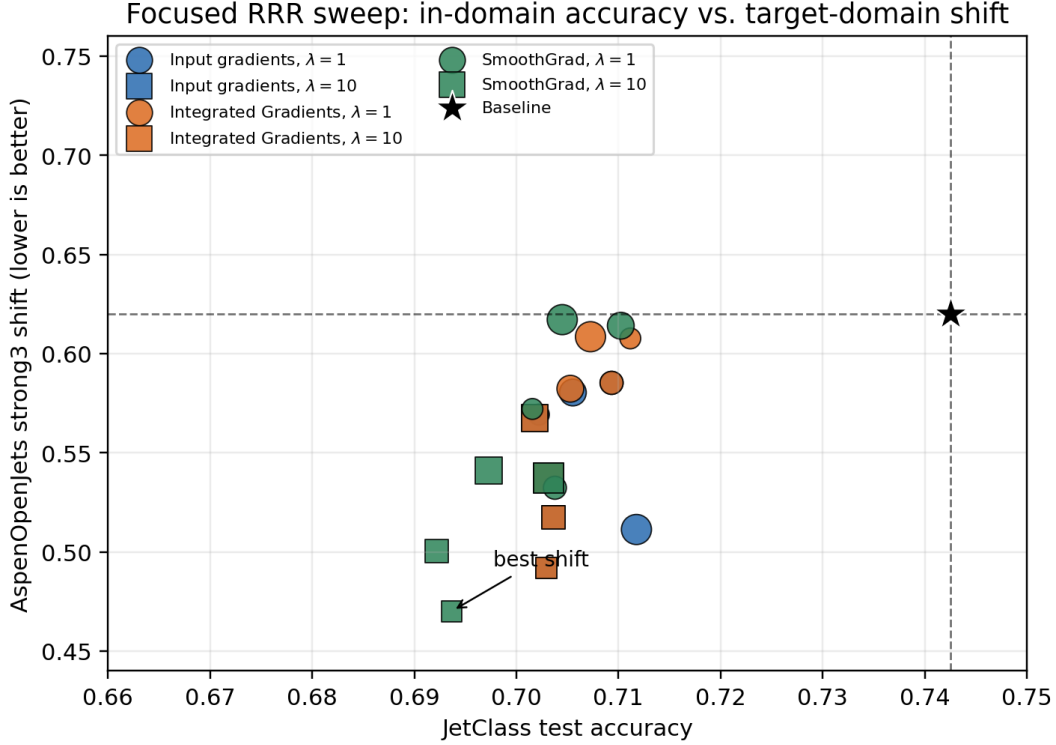


Figure 1: Focused RRR sweep configurations plotted by their JetClass test accuracy and AspenOpenJets strong3 shift. The sweep uses $\lambda_{\text{RRR}} \in \{1, 10\}$, four mask fractions, and three different attribution-mask sources. Each point is the best iteration within a hyperparameter configuration, selected by the minimum AspenOpenJets strong3. The black start is the baseline model. Point color indicates the attribution source used to build the masks, marker shape indicates λ_{RRR} , and point size increases with mask fraction.

6 Discussion

The corrected baseline confirms that a compact transformer trained on a relatively small subset of JetClass can be a strong in domain classifier, though it is below the performance of the full-scale Particle transformer results reported in the JetClass paper [1]. That is expected though: this project used 150,000 training jets per seed while the JetClass ones used 10s of millions. The goal here was not to reproduce the state-of-the-art JetClass results, but to create a controlled and reproducible setup for studying shifted and interpretability guided training.

The AspenOpenJets results show how the simulation-to-real-data performance shift is large. The strongest claim supported by that data is not that the model has a known low accuracy on AspenOpenJets since there are no labels, the more real claim is that the prediction distribution and unlabeled metrics change noticeably across datasets. The pseudo-accuracy mapping was an unsuccessful part of analysis. It was useful as a diagnostic exercise on controlled corruptions, where the true labels made calibration possible, but it was too fragile on AspenOpenJets because the target points often lay outside of the calibration range. I therefore don't report pseudo-accuracy values as the final results. The conclusion is based on the direct unlabeled shift metrics, not on a claimed AspenOpenJets accuracy.

The RRR style of intervention supports the qualitative hypothesis of the project. Across all 24

Table 7: Best focused-sweep RRR configuration for each attribution-mask source. The baseline row is included for reference.

Mask source	λ	Mask frac.	Best iter.	JetClass test acc.	Aspen strong3
Baseline	—	—	—	0.7459	0.6199
Input-gradient masks	10	0.05	1	0.7029	0.4918
Integrated-gradient masks	10	0.05	1	0.7029	0.4918
SmoothGrad masks	10	0.05	5	0.6937	0.4701

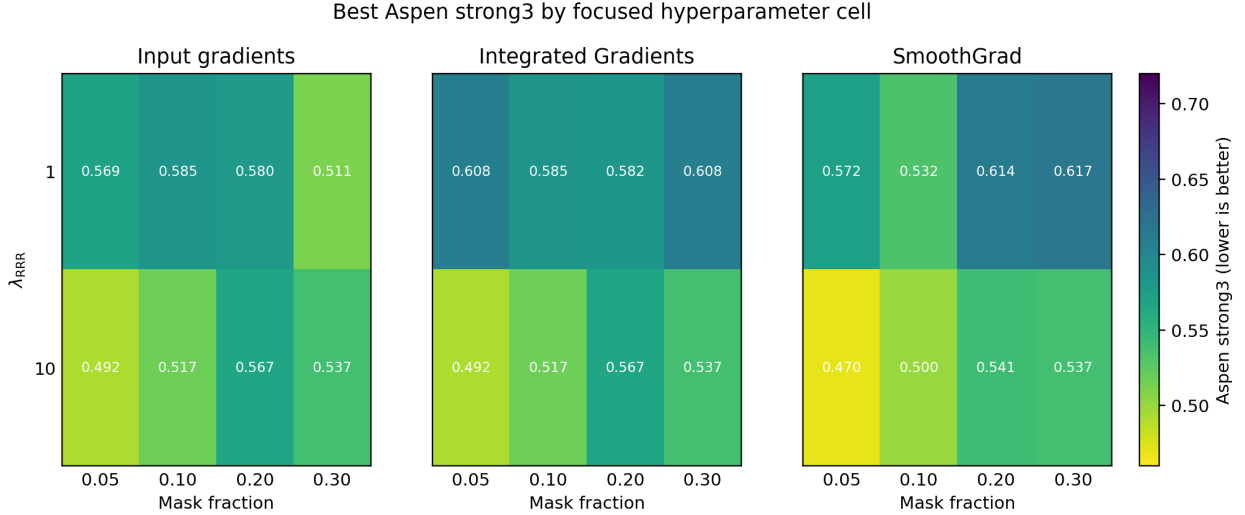


Figure 2: Best AspenOpenJets strong3 shift for each focused-sweep RRR hyperparameter cell. Each panel corresponds to the attribution source used to construct masks between find-another iterations. The displayed grid uses $\lambda_{RRR} \in \{1, 10\}$ and four mask fractions. Lower values are better.

configurations within the focused sweep, the strong3 Aspen shift summary improved relative to the baseline. The best configuration used SMOOTHGrad masks with $\lambda_{RRR} = 10$ and mask fraction 0.05 reducing strong3 from 0.6199 to 0.4701. This came with a JetClass accuracy drop from 0.7425 to 0.6937. That is a meaningful tradeoff, not just a free improvement. A less aggressive input gradient configuration of $\lambda_{RRR} = 1$ and mask fraction 0.30 gave a smaller shift reduction but preserved more JetClass accuracy. This is the kind of tradeoff the project was designed to look into.

However, this should not be overstated as a demonstrated improvement in real data classification accuracy. The strongest supported conclusion here is that the intervention reduces several calibrated prediction-distribution shift metrics between JetClass and AspenOpenJets. That is evidence in the right direction, but it is not the same as measuring actual accuracy on labeled real collision data. A fully convincing claim about improved real-test-data performance would need a larger study with a stronger target-domain validation method. However, the current real jet accuracy metrics in the field require datasets multiple terabytes in size, and I could not run nor download those dataset with my current resources.

The computational cost of the find-another strategy explains why I narrowed the hyperparameter range. A single point in the sweep corresponds to five separate “from-scratch” trainings, five JetClass test evaluations and five AspenOpenJets inference passes, and up to four full training set attribution passes for constructing the next mask.

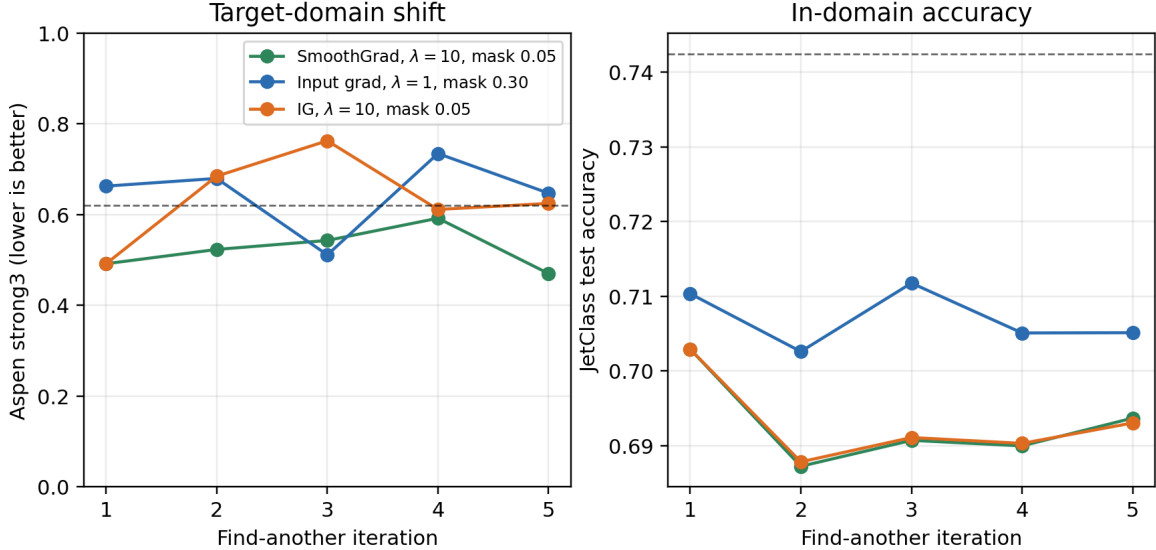


Figure 3: Find-another iteration trajectories for representative RRR configurations, one for each attribution-mask source. The left panel shows AspenOpenJets strong3 shift on a fixed 0–1 scale, and the right panel shows JetClass test accuracy for the same iterations. Dashed lines mark the baseline.

Integrated Gradients and SmoothGrad worked well for the post-training attribution but they were especially expensive inside this loop. I wasn’t able to use them directly as in-training regularizers in the same way as I used input gradients due to compute constraints. However, they were still effectively used as mask-construction methods: they identify the features after a model is trained, and those masks are then used by the next input-gradient regularized training iteration. This preserved the main scientific idea while keeping computation not too expensive.

The sanity checks were important for interpreting this limitation. They showed us that the large AspenOpenJets shift is unlikely to be explained by a simple software error such as loading the wrong checkpoint, using a different label ordering, producing invalid probabilities, or replaying corruption metrics incorrectly. There fore the failure of the pseudo-accuracy conversion should be seen as a limitation of the calibration strategy, not as evidence the pipeline itself was broken.

7 Conclusion

The final corrected analysis answers the project question in a qualified and positive way. Interpretability guided training did not eliminate the JetClass to AspenOpenJets domain shift, and it did reduce the in-domain JetClass performance. However, it very consistently moved the completed models toward lower unlabeled AspenOpenJets shift metrics. This means that the method is changing the model in the intended direction rather than merely adding noise or regularization without effect.

The main result is an accuracy robustness tradeoff measured through unlabeled target-domain diagnostics: gradient regularized, attribution guided training can reduce the apparent simulation to real data shift, but the best operating point depends on how much in-domain performance one is willing to “sacrifice”. Future work should expand the hyperparameter range beyond the focused sweep used here, repeat the best configurations over five seeds, evaluate much larger AspenOpenJets

samples, develop better target-domain validation metrics, and test methods that use unlabeled target-domain information directly during training. Another idea is to use the different ‘find another explanation’ iterations as an ensemble, as this could be the most robust use of this method.

References

- [1] Huilin Qu, Congqiao Li, and Sitian Qian. Particle transformer for jet tagging. In *International Conference on Machine Learning*, pages 18281–18292. PMLR, 2022.
- [2] Andrew J Larkoski, Ian Moulton, and Benjamin Nachman. Jet substructure at the large hadron collider: a review of recent advances in theory and machine learning. *Physics Reports*, 841: 1–63, 2020.
- [3] Roman Kogler, Benjamin Nachman, Alexander Schmidt, Lily Asquith, Emma Winkels, Mario Campanelli, Chris Delitzsch, Philip Harris, Andreas Hinzmann, Deepak Kar, et al. Jet substructure at the large hadron collider. *Reviews of Modern Physics*, 91(4):045003, 2019.
- [4] Oz Amram, Luca Anzalone, Joschka Birk, Darius A Faroughy, Anna Hallin, Gregor Kasieczka, Michael Krämer, Ian Pang, Humberto Reyes-Gonzalez, and David Shih. Aspen open jets: unlocking lhc data for foundation models in particle physics. *Machine Learning: Science and Technology*, 6(3):030601, 2025.
- [5] Jesse Thaler and Ken Van Tilburg. Identifying boosted objects with n-subjettiness. *Journal of High Energy Physics*, 2011(3):1–28, 2011.
- [6] Jesse Thaler and Ken Van Tilburg. Maximizing boosted top identification by minimizing n-subjettiness. *Journal of High Energy Physics*, 2012(2):93, 2012.
- [7] Andrew J Larkoski, Simone Marzani, Gregory Soyez, and Jesse Thaler. Soft drop. *Journal of High Energy Physics*, 2014(5):1–46, 2014.
- [8] Josh Cogan, Michael Kagan, Emanuel Strauss, and Ariel Schwartzman. Jet-images: computer vision inspired techniques for jet tagging. *Journal of High Energy Physics*, 2015(2):1–16, 2015.
- [9] Luke de Oliveira, Michael Kagan, Lester Mackey, Benjamin Nachman, and Ariel Schwartzman. Jet-images—deep learning edition. *Journal of High Energy Physics*, 2016(7):1–32, 2016.
- [10] Patrick T Komiske, Eric M Metodiev, and Jesse Thaler. Energy flow networks: deep sets for particle jets. *Journal of High Energy Physics*, 2019(1):1–46, 2019.
- [11] Huilin Qu and Loukas Gouskos. Jet tagging via particle clouds. *Physical Review D*, 101(5): 056019, 2020.
- [12] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.
- [13] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. Axiomatic attribution for deep networks. In *International conference on machine learning*, pages 3319–3328. PMLR, 2017.
- [14] Daniel Smilkov, Nikhil Thorat, Been Kim, Fernanda Viégas, and Martin Wattenberg. Smoothgrad: removing noise by adding noise. *arXiv preprint arXiv:1706.03825*, 2017.

- [15] Andrew Slavin Ross, Michael C Hughes, and Finale Doshi-Velez. Right for the right reasons: Training differentiable models by constraining their explanations. *arXiv preprint arXiv:1703.03717*, 2017.
- [16] Laura Rieger, Chandan Singh, William Murdoch, and Bin Yu. Interpretations are useful: penalizing explanations to align neural networks with prior knowledge. In *International conference on machine learning*, pages 8116–8126. PMLR, 2020.
- [17] Gabriel Erion, Joseph D Janizek, Pascal Sturmfels, Scott M Lundberg, and Su-In Lee. Improving performance of deep learning models with axiomatic attribution priors and expected gradients. *Nature machine intelligence*, 3(7):620–631, 2021.
- [18] Shai Ben-David, John Blitzer, Koby Crammer, Alex Kulesza, Fernando Pereira, and Jennifer Wortman Vaughan. A theory of learning from different domains. *Machine learning*, 79(1): 151–175, 2010.
- [19] Yaroslav Ganin and Victor Lempitsky. Unsupervised domain adaptation by backpropagation. In *International conference on machine learning*, pages 1180–1189. PMLR, 2015.
- [20] Stephan Rabanser, Stephan Günnemann, and Zachary Lipton. Failing loudly: An empirical study of methods for detecting dataset shift. *Advances in Neural Information Processing Systems*, 32, 2019.
- [21] Devin Guillory, Vaishaal Shankar, Sayna Ebrahimi, Trevor Darrell, and Ludwig Schmidt. Predicting with confidence on unseen distributions. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 1134–1144, 2021.
- [22] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, David Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *Advances in neural information processing systems*, 32, 2019.
- [23] Rochester Institute of Technology. Research computing services, 2026. URL <https://www.rit.edu/researchcomputing/>.
- [24] Jianhua Lin. Divergence measures based on the shannon entropy. *IEEE Transactions on Information theory*, 37(1):145–151, 2002.